

# Reading Assessment Calculation Guide for Flutter/Dart Implementation

## Complete Implementation Flow

### Step 1: Database Collections Overview

#### Collections Used:

1. `Pre_Assessment` - Contains assessment configuration and scoring rules
2. `user_responses` - Contains student's answers to assessment questions
3. `users` - Contains student profiles (THIS IS WHERE YOU UPDATE RESULTS)

### Step 2: Data Retrieval Functions

dart

```
// Get pre-assessment configuration (dynamic scoring rules)
Future<Map<String, dynamic>> getPreAssessmentConfig(String assessmentId) async {
  // Query: db.Pre_Assessment.findOne({"assessmentId": assessmentId})
  Map<String, dynamic> preAssessment = await database.collection('Pre_Assessment')
    .where('assessmentId', isEqualTo: assessmentId)
    .get()
    .then((snapshot) => snapshot.docs.first.data());

  return {
    'categoryCounts': preAssessment['categoryCounts'],
    'scoringRules': preAssessment['scoringRules'],
    'totalQuestions': preAssessment['totalQuestions'],
  };
}

// Get all user responses for the assessment
Future<List<Map<String, dynamic>>> getUserResponses(int studentId, String assessmentId) async {
  // Query: db.user_responses.find({"studentId": studentId, "assessmentId": assessmentId})
  QuerySnapshot snapshot = await database.collection('user_responses')
    .where('studentId', isEqualTo: studentId)
    .where('assessmentId', isEqualTo: assessmentId)
    .get();

  return snapshot.docs.map((doc) => doc.data() as Map<String, dynamic>).toList();
}
```

### Step 3: Score Calculation Functions

dart

*// Calculate score for each category*

```
int calculateCategoryScore(List<Map<String, dynamic>> responses, String categoryName) {  
  if (responses.isEmpty) return 0;
```

```
  int correctAnswers = 0;
```

```
  for (var response in responses) {  
    if (response['category'] == 'Phonological Awareness') {  
      // Special case: Use correctMatches field  
      correctAnswers += (response['correctMatches'] as int? ?? 0);  
    } else {  
      // Standard case: Use isCorrect boolean  
      if (response['isCorrect'] == true) {  
        correctAnswers++;  
      }  
    }  
  }
```

```
  return correctAnswers;  
}
```

*// Group responses by category name*

```
Map<String, List<Map<String, dynamic>>> groupResponsesByCategory(List<Map<String, dynamic>>> allResponses) {  
  Map<String, List<Map<String, dynamic>>> grouped = {};
```

```
  for (var response in allResponses) {  
    String category = response['category'];  
    if (!grouped.containsKey(category)) {  
      grouped[category] = [];  
    }  
    grouped[category]!.add(response);  
  }
```

```
  return grouped;  
}
```

## Step 4: Part 1 Score Calculation (Dynamic)

```
dart

int calculatePart1Score(Map<String, int> categoryScores, Map<String, int> categoryCounts) {
  int part1Score = 0;

  // Part 1 = All categories EXCEPT reading_comprehension
  categoryCounts.forEach((categoryKey, questionCount) {
    if (categoryKey != 'reading_comprehension') {
      // Convert category key to match response category names
      String categoryName = convertCategoryKeyToName(categoryKey);
      part1Score += categoryScores[categoryName] ?? 0;
    }
  });

  return part1Score;
}

String convertCategoryKeyToName(String categoryKey) {
  switch (categoryKey) {
    case 'alphabet_knowledge': return 'Alphabet Knowledge';
    case 'phonological_awareness': return 'Phonological Awareness';
    case 'decoding': return 'Decoding';
    case 'word_recognition': return 'Word Recognition';
    case 'reading_comprehension': return 'Reading Comprehension';
    default: return categoryKey;
  }
}
```

## Step 5: Reading Level Determination (Using Database Scoring Rules)

```
dart
```

```
String determineReadingLevel(int part1Score, int readingPercentage, int comprehensionScore, Map<String, dynamic>
```

```
// Check levels in priority order
```

```
List<String> levelOrder = ['Low Emerging', 'High Emerging', 'Developing', 'Transitioning', 'At Grade Level'];
```

```
for (String levelName in levelOrder) {
```

```
    Map<String, dynamic> rule = scoringRules[levelName];
```

```
// Check Part 1 score range
```

```
List<dynamic> part1RangeRaw = rule['part1ScoreRange'];
```

```
List<int> part1Range = [part1RangeRaw[0] as int, part1RangeRaw[1] as int];
```

```
if (part1Score < part1Range[0] || part1Score > part1Range[1]) {
```

```
    continue; // Skip this level
```

```
}
```

```
// Special case: Low Emerging only needs Part 1 score
```

```
if (levelName == 'Low Emerging') {
```

```
    return levelName;
```

```
}
```

```
// Check reading percentage range (if exists)
```

```
if (rule['readingPercentageRange'] != null) {
```

```
    List<dynamic> readingRangeRaw = rule['readingPercentageRange'];
```

```
    List<int> readingRange = [readingRangeRaw[0] as int, readingRangeRaw[1] as int];
```

```
if (readingPercentage < readingRange[0] || readingPercentage > readingRange[1]) {
```

```
    continue; // Skip this level
```

```
}
```

```
}
```

```
// Check comprehension score range (if exists)
```

```
if (rule['comprehensionCorrectRange'] != null) {
```

```
    List<dynamic> comprehensionRangeRaw = rule['comprehensionCorrectRange'];
```

```
    List<int> comprehensionRange = [comprehensionRangeRaw[0] as int, comprehensionRangeRaw[1] as int];
```

```
if (comprehensionScore < comprehensionRange[0] || comprehensionScore > comprehensionRange[1]) {
```

```
    continue; // Skip this level
```

```
}
```

```
}
```

```
// All criteria met
```

```
return levelName;
```

```

}

// Fallback
return 'High Emerging';
}

```

## Step 6: Users Table Update Function

```

dart

Future<bool> updateUserReadingResults(int studentId, String readingLevel, int readingPercentage) async {
  try {
    // Update users collection with calculated results
    await database.collection('users')
      .where('idNumber', isEqualTo: studentId)
      .get()
      .then((snapshot) {
        if (snapshot.docs.isNotEmpty) {
          String docId = snapshot.docs.first.id;
          return database.collection('users').doc(docId).update({
            'readingLevel': readingLevel,
            'readingPercentage': readingPercentage,
            'preAssessmentCompleted': true,
            'lastAssessmentDate': DateTime.now().toIso8601String(),
            'updatedAt': DateTime.now().toIso8601String(),
          });
        }
      });

    return true;
  } catch (e) {
    print('Error updating user: $e');
    return false;
  }
}

```

## Step 7: Main Execution Function

```

dart

```

```
Future<Map<String, dynamic>> processAssessmentResults(int studentId, String assessmentId) async {
  try {
    print('Starting assessment calculation for student: $studentId');

    // 1. Get dynamic assessment configuration
    Map<String, dynamic> config = await getPreAssessmentConfig(assessmentId);
    Map<String, int> categoryCounts = Map<String, int>.from(config['categoryCounts']);
    Map<String, dynamic> scoringRules = config['scoringRules'];
    int totalQuestions = config['totalQuestions'];

    print('Assessment config loaded: $totalQuestions total questions');

    // 2. Get all user responses
    List<Map<String, dynamic>> allResponses = await getUserResponses(studentId, assessmentId);

    if (allResponses.isEmpty) {
      throw Exception('No responses found for student $studentId');
    }

    print('Found ${allResponses.length} responses');

    // 3. Group responses by category
    Map<String, List<Map<String, dynamic>>> groupedResponses = groupResponsesByCategory(allResponses);

    // 4. Calculate scores for each category
    Map<String, int> categoryScores = {};
    groupedResponses.forEach((categoryName, responses) {
      categoryScores[categoryName] = calculateCategoryScore(responses, categoryName);
    });

    print('Category scores: $categoryScores');

    // 5. Calculate Part 1 score (all except reading comprehension)
    int part1Score = calculatePart1Score(categoryScores, categoryCounts);

    // 6. Calculate reading percentage
    int totalCorrect = categoryScores.values.fold(0, (sum, score) => sum + score);
    int readingPercentage = ((totalCorrect / totalQuestions) * 100).round();

    // 7. Get comprehension score
    int comprehensionScore = categoryScores['Reading Comprehension'] ?? 0;

    // 8. Determine reading level using scoring rules
```

```

String readingLevel = determineReadingLevel(part1Score, readingPercentage, comprehensionScore, scoringRu

print('Results: Level=$readingLevel, Percentage=$readingPercentage%, Part1=$part1Score');

// 9. Update users table
bool updateSuccess = await updateUserReadingResults(studentId, readingLevel, readingPercentage);

if (!updateSuccess) {
  throw Exception('Failed to update user record');
}

// 10. Return results
return {
  'success': true,
  'readingLevel': readingLevel,
  'readingPercentage': readingPercentage,
  'part1Score': part1Score,
  'totalScore': totalCorrect,
  'categoryScores': categoryScores,
};

} catch (e) {
  print('Error in processAssessmentResults: $e');
  return {
    'success': false,
    'error': e.toString(),
  };
}
}

```

## Step 8: Usage Implementation

```
dart
```

```

// Call this when student completes assessment
void onAssessmentCompleted(int studentId) async {
    // Show loading indicator
    showLoadingDialog();

    try {
        Map<String, dynamic> result = await processAssessmentResults(studentId, "1");

        // Hide loading indicator
        hideLoadingDialog();

        if (result['success']) {
            // Success - show results to user
            String level = result['readingLevel'];
            int percentage = result['readingPercentage'];

            showResultsDialog(level, percentage);

            // Navigate to next screen or update UI
            updateUIWithResults(level, percentage);

        } else {
            // Error occurred
            showErrorDialog('Failed to calculate results: ${result['error']}');
        }

    } catch (e) {
        hideLoadingDialog();
        showErrorDialog('Unexpected error: $e');
    }
}

```

## Critical Implementation Notes

### Database Field Mapping

- **users.idNumber** ↔ **user\_responses.studentId** (int)
- **users.readingLevel** ← calculated result (String)
- **users.readingPercentage** ← calculated result (int)

### Error Handling Requirements

1. Check if user responses exist before calculation



2. Validate that all required categories have responses
3. Handle null/missing scoring rules gracefully
4. Verify user exists before updating

## Testing Validation

Use existing data to verify:

- Student 202533333 should get "High Emerging" + 40%
- Student 202522233 should get "At Grade Level" + 80%
- Student 2025121 should get "Developing" + 60%

## Performance Considerations

- Cache pre-assessment config if multiple students use same assessment
- Use database transactions for user updates
- Add retry logic for database operations

This implementation handles all edge cases, provides comprehensive error handling, and updates the users table correctly with both readingLevel and readingPercentage fields.